

Documentação Técnica: Configuração e Conexão do Firebase (Authentication e Firestore)

1. Introdução

Esta documentação descreve o passo a passo realizado para integrar o Firebase ao projeto Android principal, implementando o Firebase Authentication (para controle de acesso e perfil de usuários) e o Cloud Firestore (para persistência e sincronização de dados estruturados em nuvem), atendendo aos requisitos da disciplina.

2. Configuração Inicial no Firebase Console

1. Criação do Projeto:

- Acessar o console do Firebase e criar um novo projeto para a aplicação.

2. Registro do Aplicativo Android:

- Adicionar um aplicativo Android informando o nome do pacote (*package name*) correspondente ao arquivo `build.gradle.kts` do projeto.
- Baixar o arquivo de configuração gerado, denominado `google-services.json`.

3. Ativação dos Serviços:

- Authentication: Habilitar o provedor de login do tipo Email/Password (E-mail e senha) na aba de métodos de login.
- Cloud Firestore: Criar o banco de dados em modo de teste para permitir as gravações iniciais das PoCs e entregas.

3. Integração com o Projeto Android (Gradle)

3.1. Inclusão do Arquivo de Configuração

- O arquivo `google-services.json` obtido no console foi posicionado obrigatoriamente dentro do diretório do módulo principal do app:

Plaintext

app/

├── google-services.json

└── build.gradle.kts

3.2. Configuração dos Plugins e Dependências (`build.gradle.kts`)

- **Arquivo de nível de projeto (Root):** Adicionado o plugin do Google Services de forma desacoplada (`apply false`).

- **Arquivo de nível de módulo (app/build.gradle.kts):** Ativado o plugin do Google Services no topo do arquivo:

```
plugins {  
    id("com.google.gms.google-services")  
}  
  
dependencies {  
    implementation(platform("com.google.firebase:firebase-bom:33.1.0"))  
    implementation("com.google.firebase:firebase-auth")  
    implementation("com.google.firebase:firebase-firestore")  
}
```

4. Implementação do Código (Autenticação e Firestore)

4.1. Inicialização e Telas de Cadastro/Login (Jetpack Compose)

Na `MainActivity` e nas estruturas de tela, o Firebase é inicializado e integrado para gerenciar o cadastro simultâneo de credenciais e dados de perfil:

```
// Exemplo de trecho implementado para cadastro e sincronização inicial  
  
val auth = FirebaseAuth  
val db = FirebaseFirestore  
  
auth.createUserWithEmailAndPassword(email, senha)  
    .addOnCompleteListener { task ->  
        if (task.isSuccessful) {  
            val userId = auth.currentUser?.uid ?: ""  
            val dadosUsuario = hashMapOf(  
                "nome" to nome,  
                "email" to email  
            )  
  
            // Salva os dados na coleção "usuarios" usando o UID como identificador
```

```
db.collection("usuarios")
    .document(userId)
    .set(dadosUsuario)
}
}
```

5. Considerações de Segurança

- As senhas dos usuários são gerenciadas de forma criptografada exclusivamente pelo **Firestore Authentication**, enquanto o **Cloud Firestore** armazena apenas metadados públicos do perfil (como `nome` e `email`), evitando a exposição de credenciais em texto plano no banco de dados.

6. Criação e Estruturação de Coleções no Cloud Firestore

O Cloud Firestore não utiliza tabelas relacionais tradicionais, mas sim uma estrutura baseada em **Coleções** (equivalente a tabelas) e **Documentos** (equivalente a linhas ou registros em formato JSON/Map).

6.1. Criação Manual via Console do Firebase

Para criar e validar a estrutura inicial do banco de dados diretamente pelo painel administrativo:

1. Acesse o [Console do Firebase](#) e selecione o projeto.
2. No menu lateral esquerdo, navegue até **Build > Cloud Firestore**.
3. Caso o banco de dados ainda não tenha sido inicializado, clique em **Create database**, selecione o *Modo de teste* (Test mode) para facilitar os testes iniciais e escolha a região do servidor.
4. Clique em **Start collection** (Iniciar coleção).
5. No campo **Collection ID**, defina o nome da coleção (ex.: `usuarios`).
6. Defina o primeiro documento:
 - **Document ID:** Pode ser gerado automaticamente clicando em **Auto-ID**, ou definido programaticamente pelo código (como o UID do usuário autenticado).
 - **Campos (Fields):** Adicione os atributos necessários para o domínio da aplicação (por exemplo: `nome` do tipo *string* e `email` do tipo *string*). *Nota: Evite armazenar campos sensíveis como senhas no Firestore, pois a autenticação já gerencia isso de forma segura.*
7. Clique em **Save** (Salvar).

6.2. Criação Dinâmica via Código (Android/Kotlin)

No fluxo da aplicação, não é necessário criar a coleção manualmente pelo painel. O Firestore cria a coleção automaticamente no momento em que o primeiro documento é gravado via código.

- **Exemplo de inserção de dados em uma coleção:**

```
val db = Firebase.firestore
```

```
val userId = auth.currentUser?.uid ?: ""
```

```
// Dados estruturados que compõem o documento na coleção "usuarios"
```

```
val dadosUsuario = hashMapOf(
```

```
    "nome" to nomeDigitado,
```

```
    "email" to emailDigitado
```

```
)
```

```
db.collection("usuarios")
```

```
    .document(userId) // Utiliza o UID do Auth como identificador do documento
```

```
    .set(dadosUsuario)
```

```
    .addOnSuccessListener {
```

```
        // Documento e coleção criados/atualizados com sucesso na nuvem
```

```
    }
```

```
    .addOnFailureListener { e ->
```

```
        // Tratamento de falha na gravação
```

```
    }
```